Module 06 Summary – Testing Spring Boot Applications

This module teaches you **how to test a Spring Boot application automatically**. Instead of manually opening the application and checking every feature, we write **automated tests** that verify our code quickly and consistently.

Slide 1 – Testing Spring Boot Applications

Main Idea

Before software is released, developers test it to ensure it works correctly.

Instead of manually testing every function, we use automated testing tools.

Benefits

- Detect bugs early
- Prevent new changes from breaking existing code
- Save development time
- Improve software quality

Slide 2 – What We Will Cover

The module introduces several testing tools.

Tool	Purpose
JUnit 5	Runs test methods
AssertJ	Checks whether results are correct
Mockito	Creates fake objects (mock dependencies)
@WebMvcTest	Tests only the Controller layer
MockMvc	Simulates HTTP requests without opening a browser
mvn test	Runs all tests

Slide 3 – Why Automated Tests?

Automated testing helps developers by:

1. Catching regressions

A regression happens when previously working code stops working after modifications.

Example:

Yesterday:

Deposit works.

Today another programmer changes Withdraw.

Now Deposit no longer works.

The tests immediately detect the problem.

---

2. Documenting expected behaviour

Tests explain how a method should behave.

Example:

deposit(100)

should increase the balance by RM100.

Future developers can understand the expected behaviour simply by reading the test.

---

3. Supporting refactoring

Developers often improve code structure.

If all tests still pass, the new code still behaves correctly.

---

4. Providing fast feedback

Tests usually complete within seconds.

Developers immediately know whether recent changes introduced bugs.

Two Types of Tests

Unit Test

Tests only one class.

Example:

AccountService

No Spring Boot.

No database.

Very fast.

Slice Test

Tests one layer only.

Usually the Controller.

Uses

@WebMvcTest

Slide 4 – JUnit 5

JUnit is the framework responsible for executing tests.

Common Annotations

@Test

Marks a method as a test.

@Test

void depositTest(){}

@BeforeEach

Runs before every test.

Useful for preparing test data.

@BeforeEach

void setup(){}

@DisplayName

Provides a readable test name.

Instead of

depositTest()

JUnit displays

Deposit updates account balance successfully

AssertJ

AssertJ checks whether the output matches the expected result.

Example

assertThat(balance).isEqualTo(1000);

If true

PASS

Otherwise

FAIL

---

assertThatThrownBy()

Checks whether an exception is thrown.

Useful when invalid operations should produce errors.

Example

Attempting to withdraw more money than the available balance.

Slide 5 – Mockito

Mockito creates **fake objects** called **Mocks**.

Instead of using a real database,

Mockito creates fake repositories.

Example

Normally

Service

↓

Repository

↓

Database

Testing

Service

↓

Fake Repository

No database is required.

@Mock

Creates a fake object.

@Mock

AccountRepository repository;

Initially,

every method returns

- null
- 0
- false

unless configured otherwise.

@InjectMocks

Creates the real service.

Injects fake repositories into it.

@InjectMocks

AccountServiceImpl service;

Meaning

REAL Service

↓

Fake Repository

---

when(...).thenReturn(...)

Configures mock behaviour.

Example

when(repository.findById(1L))

.thenReturn(Optional.of(account));

Meaning

Whenever someone asks for account 1,

pretend it exists.

---

verify()

Checks whether a method was called.

Example

verify(repository).save(account);

Confirms that save() was executed.

---

never()

Confirms that a method was **not** called.

Example

Duplicate account

↓

Throw exception

↓

save()

should never execute.

Slide 6 – Unit Testing AccountService

Tests should include both successful and unsuccessful scenarios.

Happy Path

Everything works correctly.

Example

Open account

↓

Save account

↓

Return account

Sad Path

An error occurs.

Example

Duplicate account number

↓

Throw IllegalArgumentException

↓

Do not save

Example

assertThatThrownBy(...)

checks whether the exception occurs.

Slide 7 – @WebMvcTest

Instead of testing the entire application,

@WebMvcTest loads only the Controller layer.

It includes

- Controller
- Filters
- Exception handlers

It excludes

- Database
- Repository
- Full Spring Boot context

@MockBean

Unlike @Mock,

@MockBean becomes a Spring Bean.

Spring injects it into the Controller automatically.

Example

Controller

↓

Mock Service

No real service implementation is used.

Slide 8 – MockMvc

MockMvc simulates HTTP requests.

Instead of opening Chrome,

MockMvc directly sends requests to the Controller.

Example

GET /api/accounts/1

Controller receives the request,

returns JSON,

MockMvc checks the response.

---

perform()

Creates the HTTP request.

Examples

perform(get(...))

perform(post(...))

---

andExpect()

Verifies the response.

Example

status().isOk()

expects

HTTP 200

---

status().isCreated()

expects

HTTP 201

---

status().isNotFound()

expects

HTTP 404

---

jsonPath()

Checks JSON fields.

Suppose the response is

```
{
  "id":1,
  "name":"Alice"
}
```

Then

jsonPath("$.id")

checks

1

HTTP Status Codes

Status	Meaning
200	Request successful
201	Resource created
400	Invalid request
404	Resource not found

Slide 9 – Running the Tests

Run all tests

mvn test

Run one test

mvn test -Dtest=AccountServiceImplTest

Test Speed Comparison

Test Type	Speed
Mockito Unit Test	Fastest
@WebMvcTest	Fast
@SpringBootTest	Slow

Reason

@SpringBootTest loads the entire application,

making it slower.

Slide 10 – Lab 06

Students are required to write tests for the Stock application.

Service Tests

- addStock()
- duplicate stock
- getAllStocks()
- getStockById()

Controller Tests

Test

GET /api/stocks

Expect

200 OK

Test

GET /api/stocks/{id}

Existing stock

200 OK

Missing stock

404 Not Found

Test

POST /api/stocks

Expect

201 Created

with

Location header

Test

Missing symbol

Expect

400 Bad Request

Overall Testing Flow

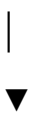
Write Test

JUnit runs the test

Mockito creates fake objects

Service or Controller executes

AssertJ / MockMvc verifies the result



PASS ✅ or FAIL ❌

Key Takeaways

- **JUnit 5** runs test methods using annotations like @Test and @BeforeEach.
- **AssertJ** makes assertions easy to read and write.
- **Mockito** isolates the class under test by replacing dependencies (such as repositories) with fake objects.
- **@Mock** creates fake objects; **@InjectMocks** injects those fakes into the real class being tested.
- **when(...).thenReturn(...)** defines how a mock should behave, while **verify()** checks whether expected methods were called.
- **Unit tests** focus on a single class and are the fastest type of test.
- **@WebMvcTest** loads only the MVC layer, making it ideal for testing controllers.
- **@MockBean** replaces Spring-managed beans with Mockito mocks inside the Spring test context.
- **MockMvc** simulates HTTP requests (GET, POST, etc.) without starting a web server.
- **mvn test** runs your project's automated tests and should be used regularly to ensure your application continues to work correctly